

Lastro: um harness de grafo com ReAct nos nós e permissão *deny-first* para gestão de marketing

Davi Eduardo

Agosto de 2026

Resumo

O gestor de conta de uma agência não faz pedidos de um passo. “Por que caíram as vendas da Ômega 3 essa semana?” exige resolver uma entidade na memória da conta, consultar duas ou três APIs, cruzar identificadores, formular e descartar hipóteses — e, no fim, defender o número numa reunião com o cliente. Este paper propõe **Lastro**, um harness híbrido para o chat agêntico da AdzHub: **grafo de estados como espinha dorsal, loops ReAct dentro dos nós de investigação** e uma **camada de permissões deny-first**, com o supercérebro da conta — grafo de entidades mais linha do tempo — tratado como fonte de contexto de primeira classe, acessada por tools dedicadas em vez de despejada no prompt. O argumento central do estudo é de enquadramento: os cinco tipos de harness de referência do desafio não estão no mesmo nível de abstração. ReAct é um loop; grafo é uma topologia; sandbox é um ambiente de execução; permissões e skills são uma camada transversal; RLM é uma estratégia de acesso a contexto. Só os dois primeiros disputam o mesmo lugar, e a pergunta certa não é “qual escolher”, é “o que fica por fora, o que fica por dentro e o que atravessa tudo”. A aposta de produto que sustenta o desenho: em marketing, a *defensibilidade* da resposta vale mais que a autonomia do agente. Dois recortes são declarados com o custo de cada um: sem sandbox/CodeAct e sem RLM completo — apenas compactação por nó. O paper nomeia onde a arquitetura quebra nas quatro tarefas reais do gestor, em vez de alegar cobertura.

Palavras-chave: harness agêntico; grafo de estados; ReAct; permissões deny-first; memória temporal; gestão de marketing.

1 Introdução

O produto-alvo da AdzHub é um chat agêntico para o profissional de marketing: o gestor descreve uma tarefa em linguagem natural e o sistema orquestra contexto e ferramentas até produzir algo útil na operação. Abaixo do chat existem três camadas: o *supercérebro* (memória da conta, um grafo de pessoas, campanhas, canais e decisões somado a uma linha do tempo), os *Apps* (metodologias da agência empacotadas: insight da semana, brief de criativo, diagnóstico de conta, análise de criativos) e as *APIs* da operação (Meta Ads, Google Ads, Analytics, CRM, WhatsApp). A conta de referência é a Housewhey, e-commerce de suplementos atendido pela

SPOT, com o time de Aline, Carolina e Luiza.

Convém separar duas coisas que a conversa pública mistura. O **modelo** é a função que, dado um contexto, escolhe a próxima ação. O **harness** é o runtime em volta: ele decide quais ações existem, o que o modelo enxerga antes de escolher, quantas vezes pode tentar, o que é feito com cada resultado, o que fica registrado e o que exige um humano antes de acontecer. A documentação do AI SDK decompõe um agente exatamente nesses três elementos — LLM, tools e loop — e é o terceiro que este paper trata [6]. Trocar o modelo melhora a média das respostas. Trocar o harness muda o que o sistema consegue prometer.

De chatbot a agente. Três propriedades fazem a diferença no dia a dia do gestor, e nenhuma delas é “responder melhor”. Primeira: o sistema **resolve entidade contra a operação real**. “A Ômega 3” vira uma ideia de campanha da Housewhey e uma janela de datas concreta, antes de qualquer chamada de API. Segunda: ele **busca o dado que ele mesmo decidiu que faltava**. Num diagnóstico de queda de vendas, ninguém programou “olhar o link de destino dos criativos”; esse passo nasce da observação de que o Meta reporta mais conversão do que o CRM atribui. Terceira: ele **age sob permissão** — pausa anúncio de verdade, depois de mostrar o que vai pausar e quanto aquilo gastou. Chatbot devolve texto; agente devolve efeito rastreável.

Tese. Proponho um harness híbrido: grafo de estados como espinha dorsal, loops ReAct dentro dos nós de investigação e permissões deny-first como camada transversal, com o supercérebro acessado por tool. O grafo dá fronteiras nomeadas — *interpret, plan, fetch, reason, gate, act, respond* — que tornam um diagnóstico de cinco etapas legível para quem vai repeti-lo numa call com o cliente. O ReAct dentro do nó preserva a liberdade de descobrir o passo que ninguém planejou, sob teto de passos e allowlist própria. O gate garante que nenhuma ação com efeito real acontece sem um humano ver o preview do efeito. O nome do harness resume a aposta: nenhuma afirmação circula sem o lastro que a segura.

Mapa. A Seção 2 fixa o vocabulário e lê o domínio. A Seção 3 é o coração argumentativo: por que grafo mais ReAct mais permissão, por que não cada uma das outras referências isoladamente, o que é tool, o que é memória, o que é App, o que mudou no meu entendi-

mento durante o estudo e quais trade-offs foram aceitos de propósito. As Seções 4 e 5 tratam do que foi efetivamente construído e de como executá-lo. A Seção 6 nomeia onde a arquitetura quebra nas quatro tarefas reais do gestor, e a Seção 7 diz o que eu construiria em seguida — e o que deliberadamente não construiria.

2 Preliminares

2.1 Harness, no vocabulário deste paper

Loop. A regra de repetição: o modelo propõe, o runtime executa, o resultado volta e o ciclo recomeça até uma condição de parada. **Tool** é uma capacidade declarada com assinatura e argumentos validados; adoto o contrato *Action-Execution-Observation* do OpenHands SDK, em que o LLM propõe um JSON, o runtime valida contra um esquema, um executor roda e o resultado retorna como *observação* [10]. **Observação** é justamente esse retorno já convertido em algo que entra no contexto — e, neste desenho, é a *única* coisa que entra. **Allowlist** é o conjunto de tools visível num dado momento; o AI SDK entrega isso por `prepareStep` com `activeTools`, sem grafo nenhum [7]. **Teto de passos** é o freio de custo: na versão 7.x do AI SDK, `stopWhen` com `isStepCount(20)` é o padrão¹ [7]. **Estado** é a estrutura tipada que sobrevive entre etapas — no LangGraph, um `State` em que cada chave pode ter um *reducer* dizendo se o update substitui ou acumula [4]. **Checkpoint** é o ponto onde esse estado é salvo; no LangGraph, nas fronteiras de *superstep* e nunca no meio de um nó. **Gate** é o ponto em que o turno para e devolve a decisão a um humano.

Vale dizer o que *não* é harness: prompt de sistema não é harness, e RAG no prompt tampouco. Ambos influenciam o modelo sem criar fronteira verificável nenhuma.

2.2 O domínio, como eu o leio

As três camadas do enunciado não são três fontes de dado equivalentes. O supercérebro é **memória**: responde “quem é quem” e “o que aconteceu quando”. Os Apps são **metodologia**: são o ativo da agência, o jeito da SPOT de diagnosticar uma conta ou escrever um brief. As APIs são **dado bruto** da operação. A Figura 1 mostra como o harness se põe entre o gestor e as três.

Mais importante: as quatro tarefas típicas do gestor **não são a mesma classe de problema**. T1, o relatório de criativos cruzado com resultado real por `utm_content`, é um *join determinístico* que precisa dar o mesmo número toda semana. T2, o diagnóstico de anomalia, é uma *investigação aberta* cujo caminho ninguém enumera antes. T3, a pauta de reunião, é *navegação de um histórico* que só cresce. T4, a análise de criativos com proposta de pausa, *termina numa ação irreversível* sobre a verba de um cliente. Um harness sintonizado para uma delas está desafinado para as outras três — e é daí que o híbrido nasce, não de indecisão.

Tabela 1: Catálogo de tools desenhado para o domínio: 10 de leitura, 2 de escrita, toda escrita atravessando o gate. A tabela é **ilustrativa** do contrato de fronteira — não é uma allowlist de produção, que seria por conta, por perfil de usuário e versionada.

Tool	Camada	Efeito	Nó
<code>graph_query</code>	supercérebro	read	int., fet., rea.
<code>timeline_query</code>	supercérebro	read	int., fet., rea.
<code>meta_ads_insights</code>	API	read	fetch
<code>google_ads_insights</code>	API	read	fetch
<code>ga_report</code>	API	read	fetch
<code>crm_leads</code>	API	read	fetch
<code>list_criativos</code>	API	read	fetch
<code>get_metrics</code>	API	read	fetch
<code>app_diagnostico</code>	App	read	reason
<code>propose_ctas</code>	App	read	reason
<code>pause_ads</code>	API	write	act (pós-gate)
<code>send_whatsapp</code>	API	write	act (pós-gate)

3 Projeto

3.1 Cinco referências, três níveis de abstração

O enunciado lista cinco tipos de harness e pede que se escolha um, combine ou invente. A primeira coisa que o estudo produziu foi a suspeita de que a lista é uma armadilha de enquadramento — não por erro do enunciado, mas porque os cinco itens não descrevem o mesmo tipo de objeto:

- **ReAct** é um *loop*: uma regra sobre como intercalar raciocínio e ação [11].
- **Grafo de estados** é uma *topologia*: quais etapas existem e como se ligam [4].
- **Sandbox/CodeAct** é um *ambiente de execução*: onde a ação roda e com que isolamento [9].
- **Permissões e skills** é uma *camada transversal*: o que é permitido e qual instrução está carregada [2].
- **RLM** é uma *estratégia de acesso a contexto*: o contexto é injetado ou navegado [12].

Só os dois primeiros disputam de fato o mesmo lugar. Os outros três compõem com qualquer coisa. A pergunta de arquitetura, portanto, não é “qual dos cinco”; são três perguntas separadas: **o que fica por fora** (a topologia), **o que fica por dentro** (o loop) e **o que atravessa tudo** (permissão, contexto, ambiente). Este paper responde: grafo por fora, ReAct por dentro, permissão deny-first atravessando; ambiente de execução recusado; contexto acessado por consulta.

A Tabela 2 resume o julgamento a que cheguei sobre os cinco tipos *puros*, aplicados a este domínio. Ela torna difícil negar três coisas. **(i)** Auditabilidade e flexibilidade são anticorrelacionadas nos tipos puros — os dois que pontuam alto em flexibilidade pontuam baixo em auditabilidade, e vice-versa; não é coincidência, já que auditabilidade vem de estrutura declarada *antes* e flexibilidade vem de estrutura decidida *durante*. **(ii)** A

¹`maxSteps` não aparece mais na documentação 7.x; o controle atual é `stopWhen`. Registro a correção porque material de 2025

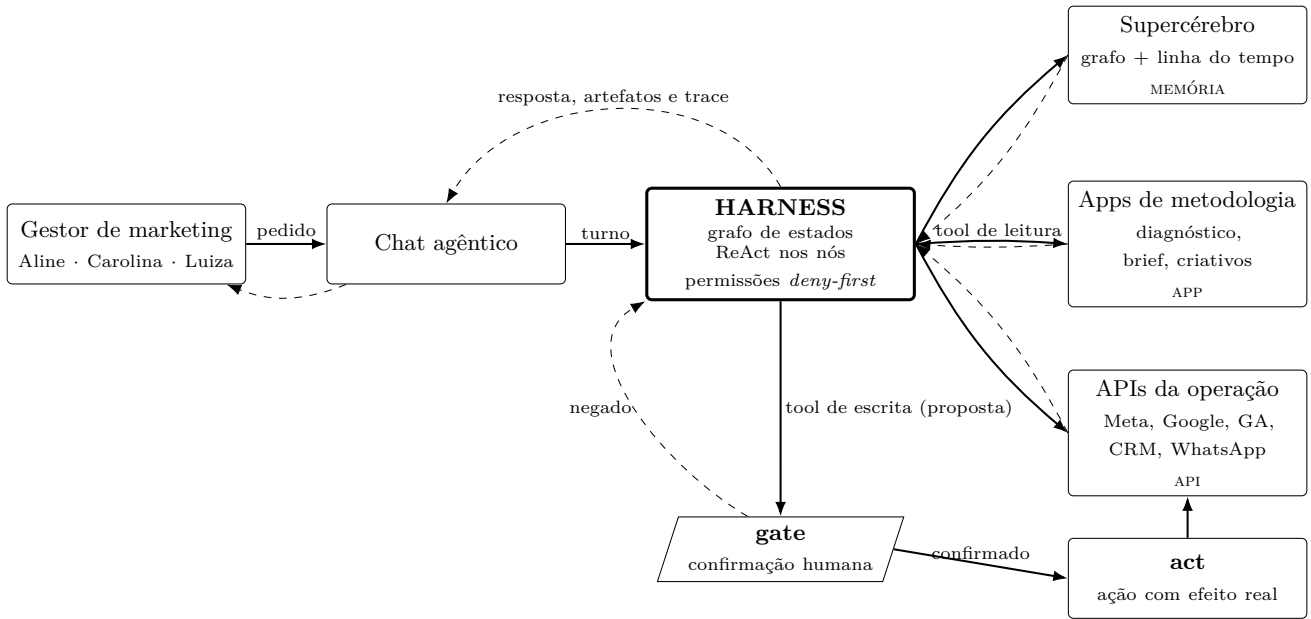


Figura 1: O harness entre o gestor e as três camadas. O **supercérebro** é **memória** (grafo de entidades mais linha do tempo); os **Apps** são **metodologia** empacotada, que devolve estrutura e não parágrafo; as **APIs** são o dado bruto da operação. As três são acessadas por um mecanismo único — **tool** — e nada entra no contexto do modelo por outro caminho. Setas cheias são chamadas do harness para fora; tracejadas são retornos que viram observação no estado. O **gate não é tool**: é um nó do grafo, que interrompe o turno e devolve um preview em português antes de qualquer escrita.

Tabela 2: Os cinco tipos *puros* avaliados para o domínio da AdzHub. Notas de 1 (ruim) a 5 (ótimo). Colunas: auditabilidade, flexibilidade, segurança operacional, custo/latência e esforço de modelagem. **É julgamento do autor** a partir das mecânicas descritas nas fontes, não medição de nenhuma delas.

Tipo	Aud.	Flex.	Seg.	Custo	Model.
ReAct	2	5	2	4	5
CodeAct / sandbox	2	5	1	3	2
Permissões & skills	3	4	5	4	3
Grafo de estados	5	2	4	4	1
RLM / ambiente	1	5	2	1	3

coluna de segurança operacional é quase ortogonal às outras, o que reforça que permissões não são um tipo concorrente e sim uma camada. (iii) Ninguém combina custo baixo com flexibilidade alta exceto o ReAct, que paga a conta na auditabilidade.

3.2 Por que nenhuma delas sozinha

Cada tipo puro foi executado mentalmente contra uma tarefa real da Housewey até quebrar, e cada falha recebeu nome. O resumo:

ReAct puro — a conclusão órfã. No diagnóstico T2 o loop roda doze ou treze chamadas e escreve um parágrafo correto. Na terça-feira, na call, a Aline pergunta “de onde saiu que a taxa de comparecimento caiu?”. O número existe na resposta e não existe em lugar nenhum do traço como resultado nomeado: foi calculado de cabeça a partir de dois JSONs que passaram pelo contexto seis passos antes. Para auditar, o gestor teria que ler as treze observações cruas — exatamente o trabalho que ele delegou. Pior: numa conversa

longa entra a compactação. O OpenHands documenta que seu **Condenser** descarta eventos e os substitui por sumários, e que é isso que reduz custo de API em até 2× [10]; num ReAct puro, sem log fora da janela, a observação que sustentava a conclusão pode ter sido resumida para fora antes da resposta final. A resposta sobrevive; a evidência, não. A interpretabilidade que o ReAct reivindica é a *do traço*, legível por quem lê o traço inteiro — não é a auditabilidade estrutural de que o gestor precisa.

CodeAct puro — a metodologia volátil. T1 é onde o CodeAct deveria ganhar de lavada, e quase ganha: cruzar gasto por anúncio com leads do CRM é literalmente a “composição de múltiplas tools” que o paper original diz que o formato JSON não faz [9]. Duas falhas aparecem depois. A primeira: a definição da SPOT do que conta como lead qualificado passa a viver no código que o modelo escreveu *naquele* turno; na semana seguinte ele escreve outro, trata os leads órfãos de outro jeito, e o cliente recebe dois relatórios diferentes sem que nada tenha mudado na conta. O App de metodologia deixa de ser App e vira uma sugestão que o modelo segue quando lembra — para uma empresa cujo produto é a metodologia empacotada, trocar App por script gerado é vender o ativo. A segunda: para fazer o join, os leads entram no processo com nome, telefone e e-mail. O OpenHands trata a versão *credencial* desse problema com um **SecretRegistry** que injeta segredos só na execução e mascara ocorrências na saída [10] — mas isso protege o token do Meta Ads, não o telefone do lead. Um sandbox de propósito geral não sabe o que é dado pessoal.

Permissões e skills sozinhas — a ordem não

governada. O fluxo de avaliação do Claude Agent SDK (hooks → deny → ask → modo → allow → *callback*) é a melhor especificação de segurança disponível de graça [2], e as *skills* do mesmo SDK resolvem elegantemente o problema dos Apps de metodologia [3]. Mas permissão governa *o quê*, nunca *quando*. Em T4, nada no sistema de permissões tem opinião sobre sequência: o agente pode listar os criativos, ler as copies e escrever três variações de CTA *antes* de olhar o desempenho, porque escrever copy é a parte que ele faz bem e a métrica é a parte chata. O resultado é um briefing bonito derivado do texto do criativo em vez do dado. Não existe regra de permissão que expresse “não proponha CTA antes de ter lido a métrica”, porque isso não é uma pergunta sobre autorização.

Grafo puro — a enumeração antecipada. Um grafo fixo (coletar → pendências → montar_pauta) resolve T3 muito bem, toda segunda-feira, de forma auditável. Aí o gestor lê e digita: “e o que ficou pendente com a Luiza no WhatsApp semana passada?”. Não há nó para isso. As opções são todas ruins: alucinar com o que está no estado, responder que não sabe fazer, ou abrir o código e acrescentar um nó — que é a resposta certa em engenharia e a errada num produto de chat, onde a pergunta seguinte é sempre imprevisível. O gestor de marketing não conversa em fluxograma, e o próprio enunciado diz que não predefine o que o chat precisa fazer.

RLM puro — latência sem teto e auditoria descartável. O princípio é o certo: em vez de injetar o contexto, deixar o modelo navegá-lo programaticamente. Os ganhos são grandes onde o corpus não cabe na janela — em OOLONG a 132k tokens, o RLM sobre GPT-5-mini atinge cerca de 64 pontos contra cerca de 30 do modelo puro, com custo por query aproximadamente igual [12]. Mas os próprios autores declaram que as chamadas recursivas são bloqueantes, que a duração vai de segundos a “vários minutos”, que não há garantia forte sobre custo total, e que a degradação é maior justamente em problemas de *contagem*. Um chat interativo tem um orçamento de paciência de dezenas de segundos, e o pior caso não é o caso médio: é o caso que acontece na frente do cliente. Some-se a isso que a procedência de um “3 decisões pendentes” passa a ser um trecho de Python que o modelo escreveu, executou e não guardou.

3.3 A escolha: grafo por fora, ReAct por dentro, permissão atravessando

Por fora, o grafo. O harness é um grafo de estados com nós nomeados e edges condicionais nomeadas (Figura 2). O estado é explícito e serializável; cada transição é um checkpoint. Três coisas passam a existir por construção. Primeira, **allowlist por fase**: *pause_ads* só existe no nó *act* — não é uma instrução de prompt que o modelo pode ignorar, é topologia. Segunda, **trace legível como narrativa**: eventos de entrada e saída de nó dão à interface a legenda *pedido* → *raciocínio* → *tool* → *observação* → *ação* → *resposta* sem que ela precise inferir nada. Terceira, **retomada**: o gate interrompe o turno de verdade, e voltar é reentrar num nó com o

estado restaurado, não recomeçar a conversa.

A objeção padrão ao grafo é que auditoria custa caro. Ela não se sustenta com números na mesa. O OpenHands mede o overhead do seu event sourcing em 39.870 eventos de 433 conversas: **0,20 ms de persistência por evento** (mediana; 0,31 ms no P95), 4,1 ms para reconstruir o estado inteiro por replay e 380 KB de armazenamento por conversa (mediana) [10]. No mesmo trabalho, o redesenho arquitetural centrado nesse log acompanhou uma **queda de 61% nos erros atribuíveis ao sistema** — de 78,0 para 30,0 por mil conversas — num rollout de produção de 15 dias com as duas versões servindo usuários em paralelo. O custo de um agente é o LLM, não o log. Quem economiza estrutura não está economizando dinheiro; está economizando a única coisa que torna a resposta defensável.

Por dentro, ReAct. Se cada nó fizesse uma chamada fixa de tool, o harness seria um workflow determinístico — e workflow determinístico não diagnostica. Dentro de *fetch* e *reason* roda um loop ReAct: pensar, chamar tool, observar, repetir, limitado por três coisas — teto de passos por nó, allowlist do nó (checada duas vezes, na configuração do nó e na definição da tool) e um teto de tokens de observação que, estourado, desvia para o nó *compact* em vez de truncar. O ciclo *fetch* ↔ *reason* tem teto próprio; ao estourar, o grafo força a saída para *respond*, que redige *declarando a lacuna* em vez de preencher com plausibilidade. É essa liberdade interna que permite o passo que ninguém planejou: buscar o link de destino dos criativos só faz sentido *depois* de observar que o Meta reporta mais conversão do que o CRM atribui.

Atravessando, a permissão. Toda tool declara efeito *read* ou *write*. Leitura roda livre dentro da allowlist. Toda escrita para o turno no nó *gate*, que monta um preview em português — itens afetados pelo nome que o gestor reconhece e com o número que justifica; impacto em uma frase; se é reversível e como desfazer; e o que acontece se ele negar — e devolve o controle. O desenho é deny-first porque a fonte mais explícita sobre isso avisa que uma checagem colocada tarde demais é silenciosamente contornada: no Claude Agent SDK, *tools auto-aprovadas nunca chegam ao callback canUseTool*, e por isso a verificação que precisa valer para toda chamada tem que morar num hook que roda antes de todo o resto [2]. A armadilha correspondente aqui é fácil de imaginar: alguém marca a leitura do Meta Ads como permitida, alguém acrescenta a pausa de anúncios ao mesmo conector, e a confirmação some. A separação do OpenHands entre *quem avalia risco* e *quem aplica a política* [10] é o que permite política diferente por conta sem reescrever tool.

Vale registrar que esse vocabulário é o da própria casa: a descrição do episódio do podcast da AdzHub sobre harness anuncia o loop ReAct, a compressão de contexto e as políticas deny-first entre seus temas [1].²

²Não obtive transcrição nem áudio do episódio; a única coisa acessível foi a descrição publicada. Cito como descrição de episódio, não como argumento técnico. O mesmo vale para o episódio

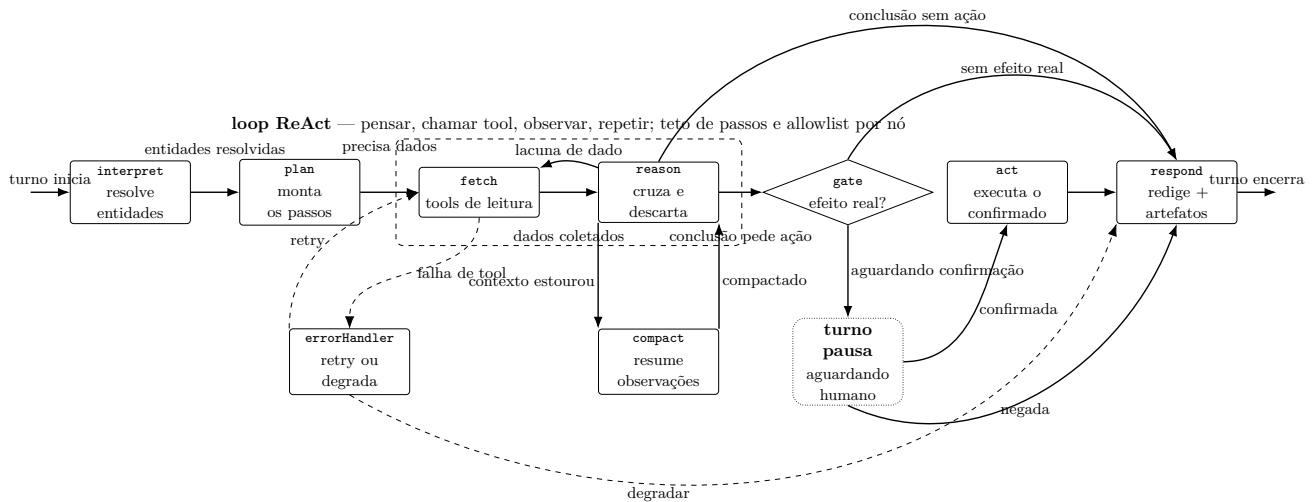


Figura 2: O grafo de estados interno. O **loop ReAct** vive apenas dentro de **fetch** e **reason** (caixa tracejada): lá o modelo escolhe tool, observa e decide o próximo passo, sob teto de passos e allowlist própria. Fora dela não há loop — os demais nós são passagem única. O **gate interrompe o turno de verdade**: a resposta HTTP fecha em “aguardando confirmação” e a execução só retoma num novo pedido com a decisão do gestor; não existe caminho de **reason** direto para **act**. Omitidas para legibilidade: as saídas diretas de **interpret** (ambiguidade de entidade: o agente pergunta em vez de chutar) e de **plan** (pedido respondível sem dado novo) para **respond**, e o caminho de erro que sai de **act**, idêntico ao de **fetch**.

3.4 Gate e efeito precisam ser nós separados

Esta é a consequência de projeto mais cara que o estudo produziu, e ela vem de uma linha da documentação do LangGraph que a maioria dos tutoriais ignora: efeitos colaterais anteriores a um `interrupt()` *devem ser idempotentes*, porque, na retomada, **o nó afetado roda de novo desde o início da função** [4]. O checkpoint salva nas fronteiras de superstep, não no meio do nó.

O que isso significa em produção de marketing é direto. Se o mesmo nó pede a aprovação e dispara o WhatsApp para a Aline, a retomada manda a mensagem duas vezes. Não existe “despausar” uma mensagem enviada ao cliente. Daí a regra não-negociável deste desenho: um nó **gate** que *só* interrompe e monta o preview, e um nó **act** que *só* executa — e que executa exatamente os argumentos que estavam no preview, sem re-inferir nada, para que o gestor não aprove uma coisa e outra aconteça. Não existe caminho de **reason** direto para **act**.

3.5 O que é tool, o que é memória, o que é App

A regra que organiza a fronteira inteira cabe em uma frase: **tudo que entra no contexto do modelo entra como retorno de tool**. Não existe contexto ambiente, não existe bloco de conta colado no prompt de sistema. Memória, App e API são *camadas*; tool é o *mecanismo único de acesso* a todas elas. A consequência prática é que cada afirmação da resposta tem um evento de trace com uma fonte citável atrás dela.

Memória é o supercérebro, acessado por `graph_query`

(navegação por tipo, id, texto ou relação) e `timeline_query` (eventos datados, filtráveis por entidade, janela e tipo). A alternativa óbvia era RAG no prompt, e ela foi rejeitada por três motivos. Custo fixo em todo turno para um contexto que a maioria dos turnos não usa. Perda de rastreabilidade — contexto colado no prompt é indistinguível de alucinação na hora em que o cliente pergunta “como você sabe disso?”. E, principalmente, similaridade vetorial responde mal a perguntas relacionais e temporais, que são o formato da maioria dos pedidos aqui: “o que mudou desde a última reunião”, “quem aprovou esse criativo”. Aqui entra o conceito que o enunciado só insinua ao dizer “linha do tempo”: a **bi-temporalidade** das arestas do Graphiti/Zep, que separa *quando o fato passou a valer* de *quando o sistema soube dele* [5]. É a diferença entre “o CPA subiu depois que trocamos o criativo” e “subiu antes, só descobrimos depois”. Numa investigação de anomalia, essa distinção é a resposta.

App é uma metodologia empacotada que devolve *estrutura*, não parágrafo: `app_diagnostico` retorna veredito, causas-raiz com evidência e fonte, hipóteses descartadas e próximos passos. É o que impede a metodologia da agência de virar improvisado do modelo, e é a diferença entre um adaptador opinativo e uma API crua — a tool certa não é `meta_ads.call(endpoint, params)`, é `meta_ads.gasto_por_criativo(conta, periodo)`, cuja definição de “gasto” é da agência e não do modelo [8].

Tool, por fim, é o contrato. A Tabela 1 lista o catálogo desenhado: dez leituras e duas escritas, cada uma amarrada aos nós de onde pode ser chamada. Duas travas independentes protegem a allowlist — o orçamento do nó e o campo de nós permitidos da própria tool — de modo que um erro de configuração de nó não abra

do *How I AI* [8], do qual li apenas as notas.

acesso à pausa de anúncios. Note que `propose_ctas` é leitura: ela produz texto novo, mas não altera nada fora do harness. Confundir “gerar” com “publicar” é exatamente como um harness passa a agir sem gate.

3.6 O que eu acreditava — e o que mudou

No início eu tratava *harness* como sinônimo de *loop*. A pergunta que eu achava central era: ReAct, grafo, CodeAct ou plan-and-execute? E eu assumia que, escolhido o loop, a qualidade do agente seria basicamente a qualidade do modelo, com o harness fazendo o papel de encanamento. Duas coisas mudaram, e uma terceira menor.

Primeira: o loop é a parte menos diferenciada do problema. Todos os cinco tipos resolvem as quatro tarefas do gestor de alguma forma. O que separa uma resposta que o gestor consegue levar para a call de uma que ele não consegue defender não é o formato do loop — é o **contrato de fronteira**: o que atravessa para o contexto do modelo, com que rastro, e o que é permitido em cada fase. Por isso o artefato mais trabalhoso desta arquitetura acabou sendo a definição de tipos — o evento de trace como união discriminada serializável, o preview de ação escrito em português, a entidade resolvida carregando um grau de confiança — e não o executor do grafo. Esses tipos *são* o harness. O loop é o resto.

Segunda, e foi um erro concreto que eu cometi no primeiro rascunho: eu ia implementar permissão como middleware em volta da execução de tool. Um wrapper que checa o efeito e decide se deixa passar. Parece limpo, e é o que a maioria dos exemplos faz. Não funciona neste domínio, por um motivo específico: um middleware pode *negar*, mas não pode **parar o turno, montar um preview legível e devolver o controle ao humano com o trabalho já feito preservado para retomar depois**. Ele só sabe dizer não. E permissão que só sabe negar empurra o produto para um dos dois extremos ruins: ou o agente vira somente leitura, ou alguém desliga a checagem. Por isso o `gate` virou nó de primeira classe do grafo, com o turno fechando em “aguardando confirmação” e a retomada acontecendo por reentrada com estado restaurado. A permissão deixou de ser uma checagem e passou a ser uma **fase do fluxo de trabalho**. Foi a mudança que mais alterou o desenho — e ela só ficou completa quando cruzei com o aviso de idempotência da Seção 3.4, que obriga a separar o gate do efeito.

Terceira, menor mas honesta: eu superestimava RAG. Achava que despejar o contexto da conta no prompt de sistema era a forma pragmática de dar memória ao agente. Ao escrever o passo a passo do diagnóstico, percebi que o valor não está em *ter* o contexto — está em conseguir dizer de onde ele veio.

3.7 Trade-offs aceitos de propósito

Latência por auditabilidade. O caminho feliz tem, por estimativa de projeto, de quatro a seis chamadas de LLM, contra duas ou três de um ReAct puro. São segundos a mais em toda pergunta, inclusive nas triviais:

o grafo cobra pedágio até em “me explica esse relatório”.

Fricção por segurança. O gate interrompe o turno em toda escrita. Pausar oito criativos em oito confirmações é pior que fazer no gerenciador; agrupar em uma reduz a fricção e aumenta a aprovação cega. O trade-off não desaparece, só muda de lado.

Cauda longa por superfície pequena. Sem CodeAct, “refaz com média móvel de sete dias” não tem tool e não tem como ser improvisado. Toda capacidade nova é um ciclo de desenvolvimento, não uma frase do gestor. Em troca, a resposta a “o que esse agente consegue fazer?” cabe na Tabela 1.

Sessão longa por simplicidade. A compactação é por nó, não por sessão: o contexto do chat cresce sem teto. É problema conhecido e não resolvido. Pior, *compact* é lossy por definição — numa conversa de trinta turnos, detalhes antigos somem e o agente não sabe que sumiram.

Fidelidade aritmética por segurança. Agregação feita pelo LLM erra em silêncio quando as linhas passam de algumas dezenas. É o custo direto de recusar o sandbox, e é o assunto da Seção 6.1.

Rigor aparente. Um grafo bonito dá a impressão de que o sistema é determinístico. Ele não é: a decisão de qual edge tomar continua sendo do modelo na maioria dos casos.

4 Sistema

[Seção preenchida na revisão final, a partir do protótipo executado. Nada é afirmado aqui antes de ser verificável.]

5 Notas de execução

[Seção preenchida na revisão final: endereço da demo, chave de API, modelos e o que exatamente é simulado.]

6 Onde a solução quebra

Uma falha por tarefa típica do gestor. São falhas do harness que eu propus, não do estado da arte.

6.1 Relatório — a soma silenciosa

Sem CodeAct, o cruzamento de gasto do Meta com leads do CRM por `utm_content` é feito pelo modelo lendo linhas. Com poucos criativos funciona. Com uma conta real de centenas de anúncios, o modelo erra soma e agrupamento — e erra *em silêncio*, porque o número errado é plausível. O agravante é específico deste caso: parte dos leads chega sem `utm_content` e simplesmente não aparece na tabela. A única coisa que impede o artefato de mentir com cara de rigor é uma nota de rodapé do tipo “12 leads chegaram sem `utm_content` e não entram nesta tabela”. Ou seja: a correção da peça mais importante do relatório depende de o nó `respond` lembrar de preencher um campo opcional. Isso não é uma garantia; é uma esperança.

6.2 Diagnóstico — o agente não sabe o que não pode ver

O harness chega à causa-raiz do diagnóstico por um motivo frágil: uma das tools de leitura expõe o link de destino do criativo. Se a pista morasse num campo que nenhuma tool expõe — uma configuração de rastreamento, um redirect, uma mudança feita direto no gerenciador — o agente **não hesitaria**. Ele concluiria “queda de demanda na semana” com a mesma estrutura de evidência e a mesma confiança, porque ausência de dado não gera sinal. Some-se o teto de ciclos: uma investigação legítima que precisasse de um ciclo a mais é cortada, e o corte por orçamento é, para o gestor, indistinguível de “investiguei e não achei mais nada”. O sinal de orçamento esgotado existe no estado; ele só ajuda se a resposta o mencionar.

6.3 Pauta — completa na aparência

A consulta à linha do tempo só conhece o que foi registrado. Na operação real, a decisão mais importante da semana costuma ter acontecido numa ligação, num áudio de WhatsApp não transcrito, ou numa mudança que a Aline fez direto no gerenciador e comentou com a Carolina no corredor. A pauta sai bem formatada, com blocos, responsáveis e prioridades — e omite exatamente o item que fará a reunião existir. É o modo de falha mais perigoso dos quatro, porque a pauta *parece* completa: o gestor entra na call confiando nela em vez de conferir. Uma pauta vazia seria menos danosa que uma pauta plausível e furada.

6.4 Criativos — o gate vira teatro, o apelido não resolve

Duas falhas na mesma tarefa. **(a)** O gate depende de o gestor ler o preview. Depois da décima confirmação, ele clica “confirmar” por reflexo, e o preview passa a ser um registro de que a informação *estava* disponível — não uma decisão informada. Agrupar confirmações reduz a fricção e aumenta a aprovação cega; manter uma por item faz o gestor preferir o gerenciador. Não há saída técnica: o trade-off só muda de lado. **(b)** A resolução de entidade quebra com apelido interno. O time chama a campanha de “a nova da Ômega”; esse alias não está no grafo; a confiança cai abaixo do limiar e o agente *pergunta*. Comportamento correto e irritante — o gestor sabe perfeitamente do que está falando e o agente parece obtuso. A correção real não é técnica: é curadoria contínua de aliases no supercérebro, trabalho que o harness não faz sozinho.

7 O que eu construiria em seguida

Avaliação antes de qualquer feature nova. Um conjunto de 20 a 30 prompts com resultado esperado e um runner determinístico medindo quatro coisas: a entidade foi resolvida corretamente; as tools chamadas foram as necessárias *e apenas elas*; a causa-raiz foi encontrada quando existia; o gate foi acionado em toda escrita. Hoje os tetos de passos são números que eu

estimei olhando quatro prompts — sem medição, todo ajuste é chute e toda regressão passa despercebida. É a primeira coisa porque as outras duas ficam impossíveis de validar sem ela.

Agregação determinística dentro das tools. Mover o join por `utm_content`, o agrupamento e o cálculo de custo por lead para dentro de uma tool que faz aritmética em código, devolvendo ao modelo o resultado agregado com a contagem explícita do que ficou de fora. Isso mata a falha da Seção 6.1 sem abrir sandbox: o código é meu, não gerado. O custo é rigidez — cada agregação nova é uma tool nova — e eu aceito, porque errar soma em silêncio é pior que não conseguir agregar de um jeito específico.

Aprendizado de alias. Quando o `interpret` pergunta e o gestor corrige (“é a Ômega 3 Prospecção”), gravar esse alias no supercérebro. Ataca a falha 6.4(b) no ponto certo: o problema é de curadoria, e a correção mais barata é aproveitar a correção que o humano já está fazendo de graça.

E o que eu deliberadamente não construiria

Sandbox/CodeAct. É a tentação óbvia depois do item anterior, e por isso precisa estar escrita aqui. Código gerado com acesso a credenciais de conta de anúncios de cliente exige isolamento de verdade — VM, rede fechada, quotas, revisão de saída — e nada disso se constrói bem em uma semana. Meio sandbox é pior que nenhum.

Multi-agente com supervisor. Um agente para Meta, outro para CRM, outro para criativos. Dobra o custo de tokens, multiplica os pontos de falha e reintroduz um grafo — só que implícito, mal especificado e mais difícil de auditar. O grafo explícito já faz esse trabalho melhor.

Escrita autônoma no supercérebro. O agente inferir fatos da conversa e gravar na memória sem confirmação. É a coisa mais perigosa da lista: memória é escrita, e memória errada é pior que dado errado, porque contamina silenciosamente todos os turnos futuros — o erro deixa de ter data e vira contexto.

Mais conectores de canal. É a feature mais fácil de vender e a que menos prova a tese. Cada conector novo é largura sem profundidade.

Um modo de autonomia configurável que permita pular o gate para ações marcadas como seguras. É exatamente o mecanismo que transforma o gate em teatro por decisão de produto, em vez de por fadiga do usuário. Se um dia existir, vem depois da telemetria que mostre quais confirmações são realmente ruído — não antes.

8 Conclusão

O trabalho de arquitetura de harness, neste domínio, não é escolher entre cinco opções: é decidir o que fica por fora, o que fica por dentro e o que atravessa tudo. Lastro responde grafo, ReAct e permissão deny-first, e

recusa por escrito o sandbox e o RLM completo, com o custo de cada recusa nomeado. A aposta que amarra as três decisões é sobre o usuário, não sobre a tecnologia: o gestor de marketing não precisa de um agente que decida sozinho — precisa de um agente cujas conclusões ele consiga defender na frente do cliente, e cujas ações ele consiga autorizar sabendo o que está autorizando.

Referências

- [1] AdzHub. Harness: A engenharia oculta da IA. *AdzHub Podcast*, Spotify, July 2026. Episódio de 16 jul., ~36 min. Consultada a descrição do episódio; transcrição não disponível. Ano não indicado na página, inferido do contexto do desafio.
- [2] Anthropic. Claude agent SDK — configure permissions. <https://code.claude.com/docs/en/agent-sdk/permissions>, 2026. Acesso em 26 ago. 2026.
- [3] Anthropic. Claude agent SDK — overview. <https://code.claude.com/docs/en/agent-sdk/overview>, 2026. Acesso em 26 ago. 2026.
- [4] LangChain. LangGraph — graph API. <https://docs.langchain.com/oss/python/langgraph/graph-api>, 2026. Acesso em 26 ago. 2026.
- [5] Preston Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory, 2025.
- [6] Vercel. AI SDK — agents: Foundations. <https://ai-sdk.dev/docs/foundations/agents>, 2026. AI SDK 7.x. Acesso em 26 ago. 2026.
- [7] Vercel. AI SDK — loop control. <https://ai-sdk.dev/docs/agents/loop-control>, 2026. AI SDK 7.x. Acesso em 26 ago. 2026.
- [8] Claire Vo. What a harness is and how to build one with the Claude Agent SDK. Podcast *How I AI*, July 2026. Episódio de 8 jul. 2026. Consultadas as notas do episódio; transcrição integral não disponível.
- [9] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [10] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Mahotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. The OpenHands software agent SDK: A composable and extensible foundation for production agents. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [12] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. Versão consultada de 11 mai. 2026; números detalhados obtidos no blog técnico do primeiro autor.